Beágyazottszoftver-fejlesztés

5. gyakorlat tematikája

Az előző gyakorlat során megnéztük, hogyan lehet egy UART perifériát megszakítás segítségével kezelni. A mostani gyakorlat során pedig kipróbáljuk, hogy hogy lehet a standard output-ot átirányítani az UART-ra, illetve a printf kiírtatást úgy megvalósítani, hogy ne blokkoljon feleslegesen sokáig.

1. Új diag.h és diag.c file létrehozása

A későbbiek során sok bővítést fogunk tenni, ezért érdemes az UART kezelést egy külön .c és .h file-ba kiszervezni. Ezt a projectre jobb gombot nyomva new/Source file / Header file-ra kattintva tudjuk megtenni.

2. UART inicializálás

Hozzuk létre a diag.c file-ban a void diag_init(void) függvényt, illetve annak a prototípusát. A feladat kiindulása a 3. gyakorlat UART projecte. Az alábbiakban kivonatolva megtalálható az inicializáláshoz szükséges minden adat. Akinek meg van a régi kódja, használhatja azt is.

A header file kezdése:

```
#ifndef DIAG_H
#define DIAG_H

void diag_init(void);

#endif
```

A következő header fájlokat kell include-olni a diag.c-ben:

```
#include "em_device.h"
#include "stdbool.h"
#include "em_cmu.h"
#include "em_usart.h"
#include "em_gpio.h"
```

Inicializálni kell az UART-ot:

```
void diag_init(void)
{
	// Enable clock for GPIO
	CMU->HFPERCLKEN0 |= CMU_HFPERCLKEN0_GPIO;

	// Set PF7 to high
	GPIO_PinModeSet(gpioPortF, 7, gpioModePushPull, 1);

	// Configure UART0
	// (Now use the "emlib" functions whenever possible.)

	// Enable clock for UART0
```

```
        CMU_ClockEnable(cmuClock_UART0, true);


        // Initialize UART0 (115200 Baud, 8N1 frame format)

        // To initialize the UART0, we need a structure to hold
        // configuration data. It is a good practice to initialize it with
        // default values, then set individual parameters only where needed.
        USART_InitAsync_TypeDef UART0_init = USART_INITASYNC_DEFAULT;

        USART_InitAsync(UART0, &UART0_init);
        // USART0: see in efm32ggf1024.h

        // Set TX (PE0) and RX (PE1) pins as push-pull output and input resp.
        // DOUT for TX is 1, as it is the idle state for UART communication
        GPIO_PinModeSet(gpioPortE, 0, gpioModePushPull, 1);
        // DOUT for RX is 0, as DOUT can enable a glitch filter for inputs,
        // and we are fine without such a filter
        GPIO_PinModeSet(gpioPortE, 1, gpioModeInput, 0);

        // Use PE0 as TX and PE1 as RX (Location 1, see datasheet (not refman))
               // Enable both RX and TX for routing
        UART0->ROUTE |= UART_ROUTE_LOCATION_LOC1;
               // Select "Location 1" as the routing configuration
        UART0->ROUTE |= UART_ROUTE_TXPEN | UART_ROUTE_RXPEN;
}
```

A paramétereket nyugodtan inicializálhatjuk a `USART_INITASYNC_DEFAULT` előre definiált tömbbel, amely történetesen pontosan az általunk kívánt beállításokat tartalmazza, így nem kell az egyes paramétereket külön-külön beállítani.

Adjuk hozzá a projekthez az UART kezelését lehetővé tevő em_usart.c függvénykönyvtárat: slcp fájl → Platform→Pheripheral→USART → install.

3. Az Stdio használata (UART0)

Az UART0 perifériát eddig direkt módon használtuk az USART_Rx() és USART_Tx() függvények segítségével. Azonban hordozhatóbb kódot kapnánk, ha helyettük a standard getchar(), putchar() függvényeket tudnánk használni. Nem beszélve pl. a printf() nyújtotta kényelemről.

Szerencsére a C nyelv standard I/O-ja átirányítható. A magas szintű, általunk használható rutinok alacsonyabb szintű függvényeket használnak, amik olyan alap funkció ellátására képesek, mint egy karakter küldése, fogadása, stb. Ha ezen függvényeket úgy valósítjuk meg, hogy pl. egy UART perifériát használjanak, akkor végeredményben pl. egy printf() is a soros portra fog írni.

Az egyes fejlesztői környezetek sokszor kínálnak valamilyen megoldást a sztenderd I/O megvalósítására, ezen a gyakorlaton egy olyan megoldást fogunk használni, amely megmutatja, hogy alacsony szinten hogyan lehetséges a feladat megoldása.

A feladat megoldása során azt használjuk ki, hogy a printf függvény az utolsó lépésben, a formátum kialakítása után a _write függvény segítségével küldi ki a karaktereket a kiválasztott perifériára (pl. UART). A _write függvény az sl_syscalls.c file-ban van definiálva. Ez egy weak típusú függvény, tehát felüldefiniálhatjuk a saját funkció megvalósításával. Esetünkben a soros portra való írás a következő módon valósítható meg:

```
int _write(int file, char *data, int len) {
	int ww;

	for (ww=0; ww<len; ww++){
		USART_Tx(UART0, data[ww]);
	}

	return len;
}
```

Ezt a függvényt is hozzuk létre a diag.c file-ban.

Ezek után a main.c-ben az <stdio.h> include-olása után az alábbinak már működnie kell (persze a dia g_init() meghívása után):

- printf("Hello!\n");

```
#include <stdio.h>
#include "diag.h"

int main(void)
{
  diag_init();

  printf("Hello Silabs \r\n");

  while(1);
}
```

4. Az UART printf küldés DMA segítségével

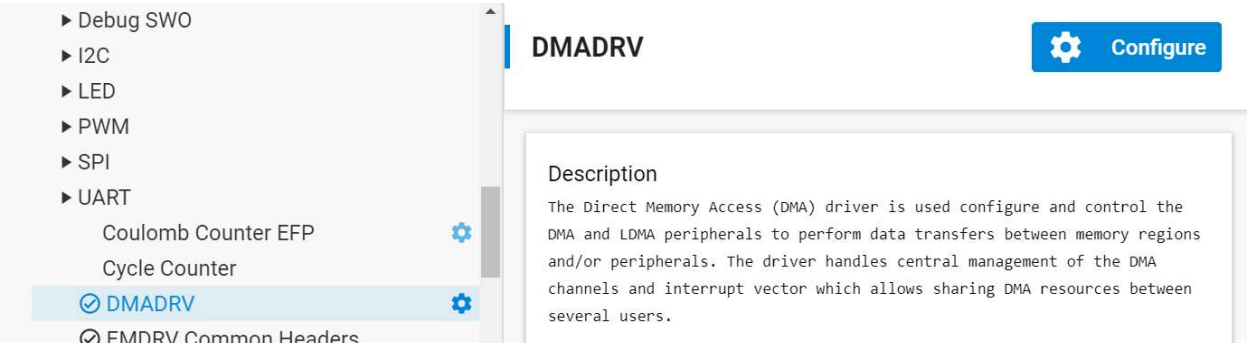
A fent bemutatott UART átirányítást nagyon sokszor használjuk a fejlesztés során debug és státusz üzenetek elküldésére. Ennek a használatnak a megfelelő megvalósításáról még az előadásokon lesz bővebben szó.

Ugyanakkor a **fent bemutatott megvalósítás hátránya, hogy blokkol**. Gyors számolással, ha szeretnénk egy 10 karakteres debug üzenetet elküldeni, akkor, mivel 1 byte elküldése 115200 baud-al hozzávetőlegesen 100µs (~50000 utasítás ciklus) ez kb. 1ms-ig fog tartani (picit rövidebb, mert van egy minimális buffer az UART-ban). Ez az idő alatt jelenleg a processzor nem csinál semmit, csak arra vár, hogy befejezzük a küldést, ami komplexebb programok esetén komoly időzítési problémákat okozhat.

Ahogy az előadáson láttuk a processzor foglaltságának csökkentésére ilyen esetekben vagy az interruptos, vagy a DMA-s periféria kezelés használható. Ebben az esetben, mivel az átvinni kívánt üzenetek általában nem 1-2 karakteresek DMA-t fogunk használni.

4.1. DMA támogatás a Silabs firmware libraryban

A .slcp konfigurációs file megnyitása után a software components tab-ban a Platform/Driver/DMADRV-t adjuk hozzá.



Miért a DMA Driver library-t adtuk hozzá, és nem az alacsony szintű DMA támogatást? Ennek a célja alapvetően az volt, hogy egy olyan hasonló konfigurációs felületet nyújtsunk, ami nem a Silabs EFM mikrovezérlőkre jellemző, hanem általánosan használható. Ez a felület hasonlít talán legjobban egy a többi vezérlőn is látható felületre, bár egyre több esetben, egyre több környezetben próbálják teljesen elfedni a DMA támogatást a legtöbb periféria esetére (A Silabs is rendelkezik olyan UART driver támogatással, amibe ez bele van építve, de mivel alapvetően a DMA-t szeretnénk gyakorolni ezt most nem használjuk). Aki szeretné kipróbálni az alacsony szintű DMA támogatást, az talál egy minta programot a LEUART DMA-n keresztüli használatára az könnyen átírható az UART0-ra is.

4.2. Milyen függvényeket tudunk használni a DMA Driverben?

A DMA driver dokumentációja, mint az összes firmware library Doxygen kommenteken alapul. A Silabs EFM Giant Gecko-jához tartozó befordított Doxygen alapú dokumentáció a következő oldalon elérhető:

https://docs.silabs.com/gecko-platform/4.5.1/platform-emlib-efm32gg/

A DMA Driverhez kötődő dokumentáció a modules rész alatt keresendő. Direkt link:

https://docs.silabs.com/gecko-platform/4.5.1/platform-driver/dmadrv

A DMA Driver bemutatása után rögtön egy mintát lehet látni, ami pont UART kommunikációra épül. Ezek alapján jól látható:

1. A *DMADRV_Init()* függvénnyel inicializálni kell a DMA drivert.
2. A DMADRV_AllocateChannel(&channel, NULL) függvénnyel le kell foglalni egy DMA csatornát. Az általunk használt vezérlőben a DMA csatornák között nem tesznek abból a szempontból különbséget, hogy milyen periféria milyen csatornát használhat, ezért a meglévő csatornák közül bármelyiket lefoglalhatjuk. Ez nem minden vezérlőcsaládban van így, például a ST STM32 vezérlőiben az egyes perifériák dedikált DMA csatornákat/stream-eket vehetnek igénybe.
3. *DMADRV_MemoryPeripheral* függvénnyel pedig elindíthatjuk az átvitelt.

4.3. Printf módosítása DMA-ra

A https://docs.silabs.com/gecko-platform/4.5.1/platform-driver/dmadrv

dokumentációban található példát (pont UART példa) könnyen át lehet úgy írni, hogy a mi UART-unkra működjön. **Vigyázzunk arra, hogy nekünk nem USART0-nk-van hanem UART0-nk.**

Az diag_init kiegészítése DMADRV_Init() és DMADRV_AllocateChannel(&channel, NULL) résszel. Ne felejtsük el a #include "dmadrv.h" -t és a unsigned int channel globális változót hozzáadni a diag.c file-hoz:

```
#include "dmadrv.h"
unsigned int channel;
```

```
  // Initialize DMA.
  DMADRV_Init();

  // Request a DMA channel.
  DMADRV_AllocateChannel( &channel, NULL );
```

Az információküldést cseréljük le a _write függvényben a paraméterek értelemszerű kicserélésével.

```
int _write(int file, char *data, int len) {

  // Start the DMA transfer.
  DMADRV_MemoryPeripheral( channel,
                           dmadrvPeripheralSignal_UART0_TXBL,
                           (void*)&(UART0->TXDATA),
                           data,
                           true,
                           len,
                           dmadrvDataSize1,
                           NULL,
                           NULL );

  return len;
}
```

Esetleg az üzenetet átírhatjuk „Hello Silabs DMA \r\n”-re (A \r\n-et érdemes használni, mert néhány printf megvalósítás képes bufferelni a küldendő dolgokat és nem rögtön küldi el, ha nem küldünk \r\n-t)

Mi a megvalósítás problémája?

Próbáljunk meg 2 prinf-et egymás után meghívni és nézzük meg, hogy mi lesz az eredménye.

Mitől van ez, és milyen az előadáson hallott szabályt szegtünk meg még, ami nem feltétlenül okozott problémát?

4.4. Egyszerű megoldás a problémára

A probléma egyszerű megoldása, hogy addig blokkoljuk a printf-elést, amíg az előző DMA átvitel nem végzett. Ez elég suta megoldás, mert esetenként blokkolni fog, de a mindig blokkoló verziónál azért jobb.

A megvalósításnál két lehetőségünk van, az egyik esetben használhatjuk a driver beépített aktivitást figyelő függvényét, vagy csinálhatunk magunknak egy saját függvényt. Mi ez utóbbit fogjuk választani, mert a későbbiekben ezt lehet majd továbbfejleszteni egy szebb megoldás felé.

A saját függvényünk alapja a DMA driver által nyújtott **callback függvény** lehetőség. A callback függvények olyan függvények, amelyeket a rendszer hív meg értesítve a programozót valamilyen eseményről. Nagyon sok esetben a callback függvények hátterében valahol egy megszakítást áll pár réteggel elfedve.

A DMA driver callback típusa az online dokumentációban megtalálható:

typedef bool(* DMADRV_Callback_t) (unsigned int channel, unsigned int sequenceNo, void *userParam)

Ebben a callback függvényben beállítunk egy diag_DMA_done flag-et, ami azt jelzi, hogy befejeződött a DMA átvitel. A _write függvényben erre a flagre várunk. Figyelni kell, hogy a _write függvény data paramétere csupán egy pointer, ami valószínűsíthetően a printf függvény lokális változója, ezért készítünk róla egy lokális másolatot, mert miután visszatérünk a _write függvényből, a printf függvény stackje leépül. Ezek alapján a _write függvény a flag változóval és a DMA callback függvényünk így néz ki:

```
volatile bool diag_DMA_done = true;
#define N_DATA 80
char loc_data[N_DATA];

int _write(int file, char *data, int len) {

  // Start the DMA transfer.
  while(!diag_DMA_done);

  diag_DMA_done = false;

  memcpy(loc_data, data, len);

  DMADRV_MemoryPeripheral( channel,
                           dmadrvPeripheralSignal_UART0_TXBL,
                           (void*)&(UART0->TXDATA),
                           loc_data,
                           true,
                           len,
                           dmadrvDataSize1,
                           diag_DMA_callback,
                           NULL );

  return len;
}
```

```
bool diag_DMA_callback(unsigned int channel, unsigned int sequenceNo, void
*userParam)
{
  diag_DMA_done = true;
  return false;
}
```

4.5. Szebb megoldás

A probléma szebb megoldása az lenne, hogyha létrehoznánk egy FIFO sort, amit a printf függvények töltenének, a DMA kiírás ürítene. A FIFO megvalósítás általában C környezetben cirkuláris buffer-el történik.

A cirkuláris buffer egy alap szoftvertechnikai megoldás, amit az előadáson is ismertettünk már. Az ilyen alap megoldásokra ahogy számos szoftver rutinra rengeteg megoldást lehet találni az interneten, ezeket érdemes használni, de mindig csak józan ésszel. Egy használható cirkuláris buffer megoldást a cirkuláris buffer leírásával a https://embedjournal.com/implementing-circular-buffer-embedded-c/ oldalon találunk. Ezt fogjuk felhasználni.

Normál esetben javasolt lenne a cirkuláris buffert is külön file-ba rakni, de most megelégszünk azzal, hogy a diag.c-file ba rakjuk.

A megvalósításhoz a cirkuláris buffer típus definicióra és a push-pop függvényre lesz szükség, illetve ezek alapján egy 128 byte-os cirkuláris buffer létrehozására.

```
typedef struct {
    uint8_t * const buffer;
    int head;
    int tail;
    const int maxlen;
} circ_bbuf_t;

#define DIAG_BUFFER_LENGTH 128
uint8_t diagBuffer[DIAG_BUFFER_LENGTH];
uint8_t diagDMASendBuffer[DIAG_BUFFER_LENGTH];
circ_bbuf_t diagCircBuffer = {diagBuffer,0,0,DIAG_BUFFER_LENGTH};
```

Átláthatóság kedvéért a push és pop függvények implementációja az 5.1 fejezetben találhatóak, illetve letölthetőek a fenti honlapról.

A cirkuláris buffert úgy fogjuk használni, hogy a printf függvények fogják feltölteni, ha éppen nincs aktív DMA átvitel, akkor el is indítják a DMA átvitelt, ha van aktív DMA átvitel, akkor csak beletöltik az új adatokat a cirkuláris buffer-be, és a DMA callback indítja el a következő átvitelt. Az átvitelhez egy globális buffer-re is szükség lesz, mert ahogy lehet látni a cirkuláris buffer-ből byte-onként juthatunk hozzá az adatokhoz (ennél lehetne hatékonyabb bufferelést is csinálni, ahol nem hozunk létre egy byte-onként kezelt cirkuláris buffert, hanem csak egy globális bufferből dolgozunk, de ez még így elég hatékony és talán szemléletesebben mutatja be, hogy mit kell ilyen esetben általánosan tenni).

Mivel az átvitelt mind a _write, mind a DMA callback függvényből el kell tudni indítani, ezért célszerű egy függvényt létrehozni erre a célra.

```
void diag_DMA_start(void)
{
  int16_t i=0;
  while( 0 == circ_bbuf_pop(&diagCircBuffer,&(diagDMASendBuffer[i])) )
      {
        i++;
      }

  if(i>0)
    {
      diag_DMA_done = false;
      // Start the DMA transfer.
      DMADRV_MemoryPeripheral( channel,
                               dmadrvPeripheralSignal_UART0_TXBL,
                               (void*)&(UART0->TXDATA),
                               diagDMASendBuffer,
                               true,
                               (i),
                               dmadrvDataSize1,
                               diag_DMA_callback,
                               NULL );
    }
}
```

Az átalakítás után a _write függvény a következő lesz:

```
int __write(int file, char *data, int len)
{
  uint16_t i=0;
  bool pushSucceded = true;

  while((i<len) && pushSucceded)
      {
        if(0 == circ_bbuf_push(&diagCircBuffer, data[i]))
          {
            i++;
          }
        else
          {
            pushSucceded = false;
          }
      }
  if(true == diag_DMA_done)
    {
      diag_DMA_start();
    }
  return i;
}
```

A callback függvény átalakítása pedig:

```
bool diag_DMA_callback(unsigned int channel, unsigned int sequenceNo, void *userParam)
{
  diag_DMA_done = true;
  diag_DMA_start();
  return false;
}
```

5. Függelék

5.1. Cirkuláris buffer push és pop műveleteinek implementációja

Az alábbi függvények a következő honlapról származnak. Egyszerűség kedvéért megtalálható az útmutatóba, hogy az önállóan használható legyen.

https://embedjournal.com/implementing-circular-buffer-embedded-c/

Függvény prototípus:

```
int circ_bbuf_push(circ_bbuf_t *c, uint8_t data);
int circ_bbuf_pop(circ_bbuf_t *c, uint8_t *data);
```

Implementáció:

```
int circ_bbuf_push(circ_bbuf_t *c, uint8_t data)
{
    int next;

    next = c->head + 1;  // next is where head will point to after this write.
    if (next >= c->maxlen)
        next = 0;

    if (next == c->tail)  // if the head + 1 == tail, circular buffer is full
        return -1;

    c->buffer[c->head] = data;  // Load data and then move
    c->head = next;             // head to next data offset.
    return 0;  // return success to indicate successful push.
}
```

```
int circ_bbuf_pop(circ_bbuf_t *c, uint8_t *data)
{
    int next;
```

```
    if (c->head == c->tail)  // if the head == tail, we don't have any data
        return -1;

    next = c->tail + 1;  // next is where tail will point to after this read.
    if(next >= c->maxlen)
        next = 0;

    *data = c->buffer[c->tail];  // Read data and then move
    c->tail = next;               // tail to next offset.
    return 0;  // return success to indicate successful push.
}
```